# Dynamic programming on choices and strings

Subset decisions and two-prefix states

FRE6883 – Financial Computation – Week 6

# The new axis is a sum, not a position

In the previous session, $i$ was the destination. Now we process items one at a time and store an answer for each sum or capacity $s$.

1. Ignore the current item: keep the old answer.
2. Take the current item: use a state at $s - x$.

To use each item at most once in a single vector, update $s$ from high to low. Then $dp[s - x]$ still belongs to the previous item prefix.

# Easy | Is an exact sum possible?

**Problem 1** Subset Sum pattern

Given nonnegative integers $a_1, \ldots, a_n$ and target $S$, determine whether a subset sums to exactly $S$. Each input position may be used at most once.

- $0 \le n \le 200$, $0 \le S \le 10^4$.
- $[3, 4, 5], S = 9 \to$ true: choose 4 and 5.
- $[3], S = 6 \to$ false: the 3 cannot be chosen twice.

# Build reachable sums by hand

Let $dp[s]$ mean: some subset of the *items already processed* sums to exactly $s$.

| Items processed | Reachable sums, up to 9 |
|---|---|
| None | $\{0\}$ |
| 3 | $\{0, 3\}$ |
| 3, 4 | $\{0, 3, 4, 7\}$ |
| 3, 4, 5 | $\{0, 3, 4, 5, 7, 8, 9\}$ |

$dp[0] =$ `true`: the empty subset has sum zero.

# Reverse order enforces the 0/1 rule

For an item $x$, the new state satisfies

$$dp_{\text{new}}[s] = dp_{\text{old}}[s] \ \vee \ \big(s \geq x \ \wedge \ dp_{\text{old}}[s - x]\big).$$

One vector can implement this by visiting $s = S, S - 1, \ldots, x$.

If we visited upward for $x = 3$, the update at $s = 3$ could immediately make $s = 6$ true. That would use the same item twice.

# Easy | Complete the C++ state update

**Complete every blank** Type, bases, bounds, and transition

```
bool subsetPossible(const std::vector<int>& a, int S) {
    std::vector<____> dp(________, ____);
    dp[____] = ____;
    for (int x : ____) {
        for (int s = ____; ________; ____) {
            dp[____] = ________ ____ ________;
        }
    }
    return ________;
}
```

Trace [3], $S = 6$, to confirm the same 3 is not reused.

### Boolean states answer existence

$O(nS)$ time, $O(S)$ space

```
bool subsetPossible(const std::vector<int>& a, int S) {
    std::vector<bool> dp(S + 1, false);
    dp[0] = true;
    for (int x : a) {
        for (int s = S; s >= x; --s) {
            dp[s] = dp[s] || dp[s - x];
        }
    }
    return dp[S];
}
```

**Invariant:** after processing $k$ items, $dp[s]$ is true exactly when a subset of those $k$ items has sum $s$.

# The STL choice makes the state explicit

- `#include <vector>` provides the DP table; `#include <algorithm>` provides `std::max` for knapsack.
- `std::vector<bool>` stores whether each sum is reachable: `true` or `false`.
- `std::vector<int>` stores counts or values in the next exercises.
- A range loop reads each item once; a descending index loop controls whether it can be reused.

**Edge cases:** an empty input can make only $S = 0$; an item larger than $S$ causes no inner-loop iteration.

**Problem 2** Same choices, more information

Given nonnegative integers $a_1, \ldots, a_n$, count subsets of input *positions* whose sum is exactly $S$.

- $0 \le n \le 30$, $0 \le S \le 10^4$: all counts fit in a 32-bit `int`.
- $[1, 2, 3, 3], S = 6 \to 3$: the pair of 3s, or 1+2 with either 3.
- Equal values at different positions are distinct choices.
- A zero may be chosen or skipped, so it doubles every current count.

# Why the boolean solution loses information

After seeing $[1, 2, 3]$, sum 3 is reachable in *two* ways: $\{3\}$ and $\{1, 2\}$. A boolean stores only `true`. Now let $dp[s]$ count subsets of the items processed so far with exact sum $s$:

$$dp_{\text{new}}[s] = dp_{\text{old}}[s] + dp_{\text{old}}[s - x].$$

The state index and descending traversal survive. The value type and combining operation change.

# Trace the number of subsets

For $[1, 2, 3, 3]$ with $S = 6$, each row lists $dp[0], \ldots, dp[6]$.

| Items | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| None | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 1, 2 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1, 2, 3 | 1 | 1 | 1 | 2 | 1 | 1 | 1 |
| 1, 2, 3, 3 | 1 | 1 | 1 | 3 | 2 | 2 | 3 |

The new 3 adds the old $dp[3] = 2$ to $dp[6] = 1$, giving 3.

# Intermediate | Complete a guided count

**Focus on the state and update**

Include the loop bounds

```
int countSubsets(const std::vector<int>& a,
                 int S) {
    std::vector<____> dp(________, ____);
    dp[____] = ____;
    for (int x : ____) {
        for (int s = ____; ________; ____) {
            dp[____] = ________ ____ ________;
        }
    }
    return ________;
}
```

Explain the empty-subset base case, then trace the second 3 in $[1, 2, 3, 3]$.

**Count choices; do not collapse them to true** $O(nS)$ time, $O(S)$ space

```
int countSubsets(const std::vector<int>& a,
                 int S) {
    std::vector<int> dp(S + 1, 0);
    dp[0] = 1;
    for (int x : a) {
        for (int s = S; s >= x; --s) {
            dp[s] = dp[s] + dp[s - x];
        }
    }
    return dp[S];
}
```

**Zero check:** $[0, 0], S = 0$ yields 4. At $s = 0$, each zero performs $dp[0] \leftarrow 2dp[0]$.

# Intermediate | Read the counting correction

Before the second 3 in $[1, 2, 3, 3]$, the table has $dp[6] = 1$ and $dp[3] = 2$.

$$dp_{\text{new}}[6] = dp_{\text{old}}[6] + dp_{\text{old}}[3] = 1 + 2 = 3.$$

- The old $dp[6]$ counts subsets that skip this 3.
- The old $dp[3]$ counts subsets that take this 3.
- Descending $s$ keeps $dp[3]$ unchanged until $dp[6]$ is computed.

A zero is a separate take-or-skip choice: it doubles every count, including $dp[0]$.

### Problem 3

0/1 knapsack

Each item $i$ has positive weight $w_i$ and nonnegative value $v_i$. Choose each item at most once. Maximize total value while total weight is at most capacity $W$.

- $0 \le n \le 200$, $0 \le W \le 10^4$; the sum of all values is below `INT_MAX`.
- $(w, v) = (3, 30), (4, 50), (5, 60)$, $W = 8 \to 90$.
- The best choice takes weights 3 and 5.

# The count is no longer the useful value

A count of subsets does not say which subset is most valuable. Define $dp[p]$ as the maximum value using processed items with *total weight at most* $p$.

$$dp_{\text{new}}[p] = \begin{cases} dp_{\text{old}}[p], & p < w, \\ \max\big(dp_{\text{old}}[p], dp_{\text{old}}[p-w] + v\big), & p \geq w. \end{cases}$$

All capacities begin at value 0 because taking no items is allowed. Visit capacities downward to preserve one use per item.

# Trace the best value at each capacity

Items arrive in order $(3, 30), (4, 50), (5, 60)$; $W = 8$.

| Items | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| None | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| $(3, 30)$ | 0 | 0 | 0 | 30 | 30 | 30 | 30 | 30 | 30 |
| $+(4, 50)$ | 0 | 0 | 0 | 30 | 50 | 50 | 50 | 80 | 80 |
| $+(5, 60)$ | 0 | 0 | 0 | 30 | 50 | 60 | 60 | 80 | 90 |

At capacity 8, take the last item: $dp_{\text{old}}[3] + 60 = 30 + 60 = 90$.

# Why the direction still matters

Suppose one item has weight 3 and value 30, and $W = 6$.

- **Descending:** when updating $p = 6$, $dp[3]$ still excludes this item. Result: 30.
- **Ascending:** after updating $p = 3$, the update at $p = 6$ may add the same item again. Wrong result: 60.

Descending capacity means 0/1 choice. Ascending capacity would model unlimited reuse, as in unbounded coin change.

# Hard | Complete the guided knapsack loop

### Initialize the state

Choose the bounds and update

```
int knapsack01(const std::vector<int>& weight,
               const std::vector<int>& value,
               int W) {
    std::vector<____> dp(________, ____);
    int n = ________;
    for (int i = ____; ________; ____) {
        for (int p = ____; ________; ____) {
            dp[____] = std::____(________,
                                 ________ + ________);
        }
    }
    return ________;
}
```

At capacity $p$, compare skipping item $i$ with taking it after a solution for capacity $p - w_i$.

# Hard | Test your implementation

| Input | Expected value |
|---|---|
| $(3, 30), (4, 50), (5, 60), W = 8$ | 90 |
| No items, $W = 8$ | 0 |
| $(3, 30), W = 6$ | 30, never 60 |
| All weights exceed $W$ | 0 |

A failing third row usually means the capacity loop runs upward.

# Hard | Correction, complete C++17 solution

```cpp
#include <algorithm>
#include <vector>

int knapsack01(const std::vector<int>& weight,
               const std::vector<int>& value,
               int W) {
    std::vector<int> dp(W + 1, 0);
    int n = weight.size();
    for (int i = 0; i < n; ++i) {
        for (int p = W; p >= weight[i]; --p) {
            dp[p] = std::max(dp[p],
                             dp[p - weight[i]] + value[i]);
        }
    }
    return dp[W];
}
```

# Hard | Read the correction at capacity 8

For the last item $(w, v) = (5, 60)$, the previous row of the table gives $dp[8] = 80$ and $dp[3] = 30$.

$$dp_{\text{new}}[8] = \max(80, 30 + 60) = 90.$$

- `dp[p]` is the value when this item is skipped.
- `dp[p - weight[i]] + value[i]` is the value when it is taken.
- Descending `p` ensures the second term uses only earlier items.

The result is $dp[W]$, since each state already means "weight at most $p$". Time $O(nW)$, space $O(W)$.

# Three questions, one update pattern

| Question | $dp[s]$ stores | Update with one item |
|---|---|---|
| Possible? | Boolean | $dp[s] \vee dp[s-x]$ |
| How many? | Number of subsets | $dp[s] + dp[s-x]$ |
| How valuable? | Best value up to capacity | $\max(dp[p], dp[p-w]+v)$ |

All three use a descending inner loop because each input position is a 0/1 choice. The requested answer determines the stored value and operation.

# Equal partition | Turn two groups into one target

## Problem 4

Reuse subset sum

Split all input positions into two groups with the same sum. Each position must belong to exactly one group; a group may be empty.

- $0 \le n \le 200$, nonnegative integers, total sum $T \le 2 \cdot 10^4$.
- $[1, 5, 11, 5] \to$ true: $11 = 1 + 5 + 5$.
- $[1, 2, 5] \to$ false: the total is even, but no subset sums to 4.
- $[1, 2] \to$ false: an odd total cannot be split equally.

If one group sums to $T/2$, what is the sum of all remaining items?

# Equal partition | Trace the reduction

Let $T$ be the sum of all elements. Equal group sums require $T$ to be even. Then ask whether a subset sums to $S = T/2$.

| Items processed | Reachable sums up to 4 for $[1, 2, 5]$ |
|---|---|
| None | $\{0\}$ |
| 1 | $\{0, 1\}$ |
| 1, 2 | $\{0, 1, 2, 3\}$ |
| 1, 2, 5 | $\{0, 1, 2, 3\}$ |

The total is 8, but 4 remains unreachable. Evenness is necessary, not sufficient. Reuse `subsetPossible` from Problem 1.

# Equal partition | Complete the reduction

## Use the earlier function

Sum, parity, and target

```
bool canPartition(const std::vector<int>& a) {
    int total = ____;
    for (int x : ____) {
        total ____ ____;
    }
    if (total ____ ____ ____ ____) {
        return ____;
    }
    return __________(____, ________);
}
```

Fill each blank with an expression, operator, or function name. Test an odd total, an even but impossible total, and empty input.

# Equal partition | Correction and edge cases

**Requires subsetPossible from Problem 1** $O(n(S+1))$ time, $O(S+1)$ space

```
bool canPartition(const std::vector<int>& a) {
    int total = 0;
    for (int x : a) {
        total += x;
    }
    if (total % 2 != 0) {
        return false;
    }
    return subsetPossible(a, total / 2);
}
```

**Why it works:** a subset of sum $S = T/2$ leaves a complement of sum $T - S = S$. Empty input and all-zero input return true; both groups may have sum zero.

**Problem 5** Unlimited reuse changes the order

Given distinct positive coin denominations and an amount $A$, return the fewest coins needed to make exactly $A$, or $-1$ if impossible. Each denomination may be used any number of times.

- $0 \le k \le 100$, $0 \le A \le 10^4$, denominations from 1 to $10^4$.
- Coins $[1, 3, 4]$, $A = 6 \to 2$: use $3 + 3$.
- Greedy takes $4 + 1 + 1$, which uses 3 coins.
- Coins $[2]$, $A = 3 \to -1$; any coin list with $A = 0 \to 0$.

# Coin change | Inspect the last coin

Let $dp[s]$ be the minimum number of coins for exact amount $s$.

$$dp[0] = 0, \qquad dp[s] = \min_{c \le s} \big( dp[s-c] + 1 \big)$$

Only reachable predecessors are candidates. Initialize other states to $\mathrm{INF} = A + 1$: a reachable amount needs at most $A$ positive coins.

| Amount $s$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| $dp[s]$, coins $[1, 3, 4]$ | 0 | 1 | 2 | 1 | 1 | 2 | 2 |

Fill amounts upward. At 6, taking coin 3 uses $dp[3] + 1 = 2$; this deliberately allows the same denomination again.

# Coin change | Complete the minimum-cost function

## Choose the unreachable marker and order

No greedy shortcut

```
int minCoins(const std::vector<int>& coins, int A) {
    const int INF = ________;
    std::vector<int> dp(________, ____);
    dp[____] = ____;
    for (int s = ____; ________; ____) {
        for (int c : ____) {
            if (________ && dp[________] != ____) {
                dp[____] = std::____(________,
                                     ________ + ____);
            }
        }
    }
    return dp[____] == ____ ? ____ : ________;
}
```

Explain why `s - c` must be checked before indexing, and why smaller amounts must be ready first.

# Coin change | Correction

**Headers: <vector> and <algorithm>** $O((k+1)(A+1))$ time, $O(A+1)$ space

```
int minCoins(const std::vector<int>& coins, int A) {
    const int INF = A + 1;
    std::vector<int> dp(A + 1, INF);
    dp[0] = 0;
    for (int s = 1; s <= A; ++s) {
        for (int c : coins) {
            if (c <= s && dp[s - c] != INF) {
                dp[s] = std::min(dp[s], dp[s - c] + 1);
            }
        }
    }
    return dp[A] == INF ? -1 : dp[A];
}
```

**Invariant:** before amount $s$, every smaller amount is solved optimally. Removing the last coin leaves one of those states.

# Coin change | Test reuse and impossible amounts

| Coins | Amount | Result | What it checks |
|---|---|---|---|
| $[1, 3, 4]$ | 6 | 2 | Greedy is not optimal |
| [3] | 6 | 2 | Reuse is allowed |
| [2] | 3 | $-1$ | Unreachable state |
| [] | 0 | 0 | Empty solution |
| [] | 5 | $-1$ | No usable denomination |

For 0/1 subsets, descend within each item to prevent reuse. Here, ascend through amounts and try every last coin: reuse is part of the problem.

# Strings use two prefix lengths

For strings $a$ and $b$, let $dp[i][j]$ describe a result for the first $i$ characters of $a$ and the first $j$ characters of $b$. The empty prefixes form row 0 and column 0. A state may look left, up, or diagonally to smaller prefixes.

`std::string` holds the input; `std::vector<std::vector<int>>` holds the table. Prefix length $i$ ends at character `a[i - 1]`.

## Problem 6

Two-prefix states

Find the maximum length of a sequence of characters appearing in order in both strings; characters need not be adjacent. For $a = \texttt{abc}$, $b = \texttt{ac}$:

| | empty | a | ac |
|---|---|---|---|
| empty | 0 | 0 | 0 |
| a | 0 | 1 | 1 |
| ab | 0 | 1 | 1 |
| abc | 0 | 1 | **2** |

If the last characters match, use the diagonal state plus 1. Otherwise skip one last character and keep the larger result.

Assume $|a|, |b| \leq 200$; either string may be empty.

# Longest common subsequence | Fill the loops

## State: LCS length of two prefixes

Choose bounds and update

```
int lcsLength(const std::string& a,
              const std::string& b) {
    int n = ________;
    int m = ________;
    std::vector<std::vector<int>> dp(
        ________, std::vector<int>(________, ____));
    for (int i = ____; ________; ____) {
        for (int j = ____; ________; ____) {
            if (a[____] ____ b[____])
                dp[i][j] = _________________;
            else
                dp[i][j] = std::____(______, ______);
        }
    }
    return dp[____][____];
}
```

# Longest common subsequence | Correction

```
int lcsLength(const std::string& a,
              const std::string& b) {
    int n = a.size();
    int m = b.size();
    std::vector<std::vector<int>> dp(
        n + 1, std::vector<int>(m + 1, 0));
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = std::max(dp[i - 1][j],
                                    dp[i][j - 1]);
        }
    }
    return dp[n][m];
}
```

# LCS | Read the correction at the last cell

At $dp[3][2]$, the final characters of `abc` and `ac` are both `c`.

$$dp[3][2] = dp[2][1] + 1 = 1 + 1 = 2.$$

For unequal characters, the correction compares skipping the last character of $a$ with skipping the last character of $b$.

Row 0 and column 0 stay zero. Time $O(nm)$, space $O(nm)$; empty strings work without a special return.

**Problem 7** Minimum cost on two prefixes

Find the fewest insertions, deletions, and replacements needed to transform $a$ into $b$. Each operation costs 1. The LCS table measures matched length; it does not keep the cost of a replacement. We need a minimum-cost state. $dp[i][j]$ is the minimum cost for the first $i$ characters of $a$ and first $j$ of $b$. If the last characters differ, choose among:

$$1 + \min\big(dp[i-1][j],\ dp[i][j-1],\ dp[i-1][j-1]\big).$$

If they match, use $dp[i-1][j-1]$ without paying 1. Example: `cat`→`cut` costs 1.

Assume $|a|, |b| \leq 200$. Empty strings are valid inputs.

# Edit distance | Read a small table before coding

For $a = \texttt{ab}$, $b = \texttt{ac}$, the completed table is:

| | empty | a | ac |
|---|---|---|---|
| empty | 0 | 1 | 2 |
| a | 1 | 0 | 1 |
| ab | 2 | 1 | **1** |

Row 0 inserts characters into the empty string. Column 0 deletes characters to reach the empty string. The final 1 replaces b with c.

# Edit distance | Complete the boundaries

## Rebuild the minimum-cost update

Dimensions, bases, bounds, and neighbors

```
// Inside editDistance(a, b): lengths n and m.
std::vector<std::vector<int>> dp(
    ________, std::vector<int>(________));
for (int i = ____; ________; ____) dp[i][0] = ____;
for (int j = ____; ________; ____) dp[0][j] = ____;
for (int i = ____; ________; ____) {
    for (int j = ____; ________; ____) {
        if (a[____] ____ b[____])
            dp[i][j] = __________;
        else
            dp[i][j] = ____ + std::____({
                __________, __________, __________});
    }
}
return dp[____][____];
```

# Edit distance | Correction

```cpp
int editDistance(const std::string& a, const std::string& b) {
    int n = a.size();
    int m = b.size();
    std::vector<std::vector<int>> dp(
        n + 1, std::vector<int>(m + 1));
    for (int i = 0; i <= n; ++i) dp[i][0] = i;
    for (int j = 0; j <= m; ++j) dp[0][j] = j;
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (a[i-1] == b[j-1])
                dp[i][j] = dp[i-1][j-1];
            else
                dp[i][j] = 1 + std::min({
                    dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
        }
    }
    return dp[n][m];
}
```

# Edit distance | Read the three neighbors

For mismatched last characters at $(i, j)$:

- $dp[i-1][j]+1$: delete the last character of $a$.
- $dp[i][j-1]+1$: insert the last character of $b$.
- $dp[i-1][j-1]+1$: replace the last character of $a$.

The table is filled from short prefixes to long prefixes, so all three neighbors are ready. Time $O(nm)$, space $O(nm)$.

# Recognize the state shape in other DP families

| Family | Typical state | Last decision to inspect |
|---|---|---|
| Paths | $dp[i]$ | Previous position |
| Subsets | $dp[s]$ | Take or skip current item |
| Grids | $dp[r][c]$ | Arrive from above or left |
| Strings | $dp[i][j]$ | Match the current characters or skip one |
| Intervals | $dp[l][r]$ | Choose a split or last action |

The index shape changes; the four questions remain: state, transition, base case, order.

# Exit ticket | Make replacement more expensive

In edit distance, insertion and deletion still cost 1, but replacement now costs 2. What changes for a mismatch?

# Exit ticket | Make replacement more expensive

In edit distance, insertion and deletion still cost 1, but replacement now costs 2. What changes for a mismatch? **Correction:**

$$dp[i][j] = \min\big(dp[i-1][j] + 1,\ dp[i][j-1] + 1,\ dp[i-1][j-1] + 2\big).$$

For matching characters, keep $dp[i-1][j-1]$. Row 0 and column 0 do not change.